

RAPPORT DE TRAVAUX PRATIQUES

Cybersécurité – Défense Périmétrique

Mise en place d'une architecture DMZ via GNS3 et Docker

Réalisé par : Aina Ny Antsa

Août 2026

Table des matières

1. Introduction.....	3
2. Méthodologie.....	3
2.1 Environnement technique.....	3
2.2 Image Docker personnalisée.....	3
2.3 Topologie réseau.....	4
2.4 Plan d'adressage IP.....	4
3. Configuration du pare-feu (iptables).....	4
3.1 Politique de filtrage.....	4
3.2 Règle NAT (masquage d'adresse).....	5
3.3 Redirection transparente vers le proxy Squid.....	5
3.4 Justification des règles.....	5
3.5 Tests de validation.....	5
4. Filtrage de contenu web (Squid).....	6
4.1 Fichier de configuration squid.conf.....	6
4.2 Liste noire des sites pour adultes.....	6
4.3 Tests de validation.....	6
5. Limitation de bande passante.....	7
6. Surveillance IDS avec Snort.....	8
6.1 Fichier de configuration snort.conf.....	8
6.2 Règles locales (local.rules).....	8
6.3 Commande de démarrage.....	8
6.4 Simulation d'un scan de ports et journal d'alerte.....	8
6.5 Analyse de l'alerte.....	8
7. Résultats des tests — synthèse.....	9
8. Analyse et conclusion.....	10
8.1 Analyse.....	10
8.2 Conclusion.....	10
Annexe — Fichiers de configuration complets.....	11

1. Introduction

Ce rapport présente la réalisation d'un Travail Pratique de cybersécurité portant sur la défense périmétrique d'un système d'information, à travers la mise en place d'une architecture réseau segmentée avec zone démilitarisée (DMZ). L'objectif est de simuler le réseau d'une entreprise exposant un service public (serveur web) tout en protégeant son réseau interne des menaces externes.

L'ensemble de la topologie a été réalisé avec GNS3, en utilisant des conteneurs Docker comme nœuds du réseau (pare-feu, serveur web, postes clients). Le pare-feu a été configuré avec iptables (filtrage et NAT), complété par Squid pour le filtrage de contenu web, tc/HTB pour la limitation de bande passante, et Snort pour la détection d'intrusion (IDS).

Ce document détaille la méthodologie suivie, les configurations mises en place, les tests de validation réalisés à chaque étape, ainsi que l'analyse des résultats obtenus.

2. Méthodologie

2.1 Environnement technique

Le TP a été réalisé sous un système d'exploitation Linux (Ubuntu), avec les outils suivants :

- GNS3 (version 2.2.61) — création et simulation de la topologie réseau
- Docker — hébergement des nœuds de la topologie (conteneurs légers, sans GNS3 VM puisque l'hôte est déjà sous Linux)
- iptables — filtrage de paquets et NAT sur le pare-feu
- Squid (5.9) — proxy web et filtrage de contenu
- Snort (2.9.15.1) — système de détection d'intrusion (IDS)
- tc (traffic control / HTB) — limitation de bande passante
- Nginx — serveur web hébergé dans la DMZ

2.2 Image Docker personnalisée

L'image Ubuntu 22.04 standard ne contenant aucun outil réseau par défaut (ni ip, ni iptables, ni ping), et les conteneurs étant isolés d'Internet une fois intégrés à la topologie GNS3, une image Docker personnalisée a été construite en amont, directement sur la machine hôte, avec tous les outils nécessaires pré-installés :

```
FROM ubuntu:22.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y \
    iproute2 \
    iptables \
    net-tools \
    iputils-ping \
    nano \
    tcpdump \
    curl \
```

```
squid \
snort \
apache2-utils \
&& apt-get clean
```

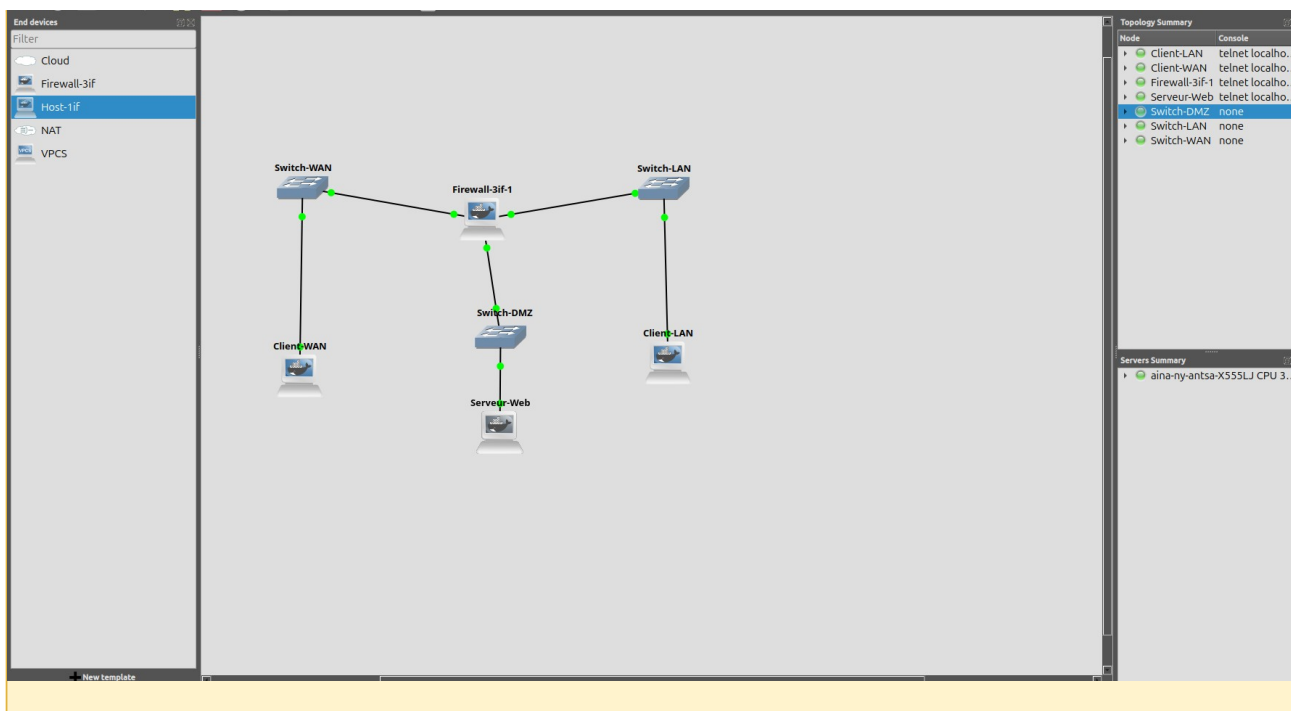
CMD ["/bin/bash"]

Cette image (dmz-node:latest) a servi de base à deux templates GNS3 : un pour le pare-feu (3 interfaces réseau) et un pour les hôtes (1 interface réseau), tous deux basés sur des conteneurs Docker.

2.3 Topologie réseau

La topologie reproduit fidèlement le schéma DMZ demandé : un pare-feu à 3 interfaces (WAN, DMZ, LAN), chaque sous-réseau étant relié par un switch Ethernet virtuel GNS3.

Sous-réseau	Plage d'adresses	Rôle
WAN	192.168.1.0/24	Simule l'accès Internet externe
DMZ	192.168.2.0/24	Héberge le serveur web (Nginx) exposé au public
LAN	192.168.3.0/24	Réseau interne de l'entreprise (postes clients)



2.4 Plan d'adressage IP

Machine	Interface	Adresse IP	Passerelle
Pare-feu (Firewall-3if-1)	eth0 (WAN)	192.168.1.1/24	—
Pare-feu (Firewall-3if-1)	eth1 (DMZ)	192.168.2.1/24	—

Pare-feu (Firewall-3if-1)	eth2 (LAN)	192.168.3.1/24	—
Client-WAN	eth0	192.168.1.10/24	192.168.1.1
Serveur-Web (DMZ)	eth0	192.168.2.10/24	192.168.2.1
Client-LAN	eth0	192.168.3.10/24	192.168.3.1

Le routage IP (ip_forward) a été activé sur le pare-feu pour permettre le transit du trafic entre les trois sous-réseaux avant toute mise en place de règle de filtrage.

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

3. Configuration du pare-feu (iptables)

3.1 Politique de filtrage

La stratégie retenue suit le principe de moindre privilège : toutes les règles d'autorisation (ACCEPT) explicites sont ajoutées en premier, et testées, avant de verrouiller la politique par défaut de la chaîne FORWARD à DROP. Cette approche évite de se couper l'accès en cours de configuration.

```
# Autoriser le trafic déjà établi (retour des connexions)
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

# Autoriser HTTP/HTTPS/SSH depuis le WAN vers la DMZ
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 443 -j ACCEPT
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 22 -j ACCEPT

# Autoriser HTTP/HTTPS/SSH depuis le LAN vers la DMZ
iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 443 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 22 -j ACCEPT

# Autoriser le LAN à sortir vers le WAN (Internet)
iptables -A FORWARD -i eth2 -o eth0 -j ACCEPT

# Bloquer explicitement le WAN vers le LAN
iptables -A FORWARD -i eth0 -o eth2 -j DROP

# Verrouillage final : tout le reste est bloqué par défaut
iptables -P FORWARD DROP
```

3.2 Règle NAT (masquage d'adresse)

Conformément au sujet, le LAN sort vers l'extérieur via NAT (masquage de l'adresse source par l'adresse du pare-feu) :

```
iptables -t nat -A POSTROUTING -o eth0 -s 192.168.3.0/24 -j MASQUERADE
```

3.3 Redirection transparente vers le proxy Squid

Afin que le filtrage de contenu (section 4) s'applique automatiquement à tous les postes du LAN sans configuration manuelle de proxy sur chaque client, le trafic HTTP sortant du LAN est intercepté et redirigé vers Squid :

```
iptables -t nat -A PREROUTING -i eth2 -p tcp --dport 80 -j REDIRECT --to-port 3128
```

3.4 Justification des règles

- ESTABLISHED,RELATED en première règle : indispensable pour laisser transiter les réponses aux connexions déjà autorisées (sinon aucune communication bidirectionnelle n'est possible, même sur les flux permis).
- Séparation WAN→DMZ et LAN→DMZ : permet d'autoriser précisément les mêmes services (HTTP/HTTPS/SSH) aux deux origines, sans ouvrir davantage que nécessaire.
- DROP explicite eth0→eth2 : bien que la politique par défaut soit déjà DROP, cette règle rend la restriction WAN→LAN explicite et documentée, indépendamment de la politique globale.
- Absence de règle ACCEPT pour eth1 (DMZ) en sortie : aucune règle n'autorise le trafic initié depuis la DMZ vers le WAN ou le LAN — combinée à la politique DROP par défaut, cela empêche le serveur web d'initier la moindre connexion sortante, conformément à l'exigence du sujet.
- MASQUERADE plutôt que SNAT statique : adapté à un environnement où l'adresse IP publique du pare-feu peut varier, et standard pour ce type de topologie.

3.5 Tests de validation

Test 1 — WAN ne peut pas accéder au LAN :

```
root@Client-WAN:/# ping -c 3 192.168.3.10
PING 192.168.3.10 (192.168.3.10) 56(84) bytes of data.
--- 192.168.3.10 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2048ms
```

Test 2 — Le serveur web ne peut sortir ni vers le WAN, ni vers le LAN :

```
root@Serveur-Web:/# ping -c 3 192.168.1.10
3 packets transmitted, 0 received, 100% packet loss
root@Serveur-Web:/# ping -c 3 192.168.3.10
3 packets transmitted, 0 received, 100% packet loss
```

Test 3 — Accès HTTP WAN → DMZ (page réelle du serveur Nginx) :

```
root@Client-WAN:/# curl -v --connect-timeout 3 http://192.168.2.10
< HTTP/1.1 200 OK
< Server: nginx/1.18.0 (Ubuntu)
<h1>Welcome to nginx!</h1>
```

4. Filtrage de contenu web (Squid)

Squid a été déployé sur le pare-feu en tant que proxy transparent (mode intercept), afin de filtrer le trafic HTTP du LAN selon deux critères demandés par le sujet : catégorie de site (sites pour adultes) et plage horaire (réseaux sociaux pendant les heures de travail).

4.1 Fichier de configuration squid.conf

```
http_port 3129
http_port 3128 intercept

acl localnet src 192.168.3.0/24

# Plages horaires de travail
acl heures_bureau_matin time MTWHF 08:00-12:00
acl heures_bureau_apresmidi time MTWHF 14:00-18:00

# Réseaux sociaux à bloquer en heures de bureau
acl reseaux_sociaux dstdomain .facebook.com .youtube.com

# Sites pour adultes (liste noire)
acl sites_adultes dstdomain "/etc/squid/blacklist_adultes.txt"

http_access deny sites_adultes
http_access deny reseaux_sociaux heures_bureau_matin
http_access deny reseaux_sociaux heures_bureau_apresmidi

http_access allow localnet
http_access allow localhost
http_access deny all
```

Remarque : le port 3129 (non-intercept) est nécessaire au fonctionnement interne de Squid (génération de ses pages d'erreur) en complément du port 3128 utilisé en interception transparente.

4.2 Liste noire des sites pour adultes

Conformément au sujet (« SquidGuard ou autre blacklist »), le blocage des sites pour adultes a été implémenté via une ACL Squid de type dstdomain pointant vers une liste de domaines, une approche fonctionnellement équivalente à SquidGuard pour cet environnement de simulation isolé :

```
.xxx
.adult-example.com
.playboy.com
```

4.3 Tests de validation

Test 1 — Accès à un domaine "réseau social" hors heures de bureau (autorisé) :

Une entrée statique a été ajoutée dans /etc/hosts du client LAN pour simuler la résolution DNS de facebook.com vers le serveur web interne (le réseau de simulation étant isolé d'Internet).

```
root@Client-LAN:/# curl -v http://www.facebook.com
< HTTP/1.1 200 OK
< X-Cache: MISS from Firewall-3if-1
< Via: 1.1 Firewall-3if-1 (squid/5.9)
```

Test 2 — Accès au même domaine en heures de bureau (bloqué) :

Les horaires ont été temporairement élargis pour inclure l'heure du test, afin de valider le mécanisme sans attendre la vraie plage horaire ; les horaires définitifs 08:00-12:00 / 14:00-18:00 ont ensuite été restaurés.

```
root@Client-LAN:/# curl -v http://www.facebook.com
<body id=ERR_ACCESS_DENIED>
<h2>The requested URL could not be retrieved</h2>
<p><b>Access Denied.</b></p>
Generated by Firewall-3if-1 (squid/5.9)
```

Test 3 — Accès à un site de la liste noire (bloqué en permanence) :

```
root@Client-LAN:/# curl -v http://www.playboy.com
<body id=ERR_ACCESS_DENIED>
<p><b>Access Denied.</b></p>
Generated by Firewall-3if-1 (squid/5.9)
```

5. Limitation de bande passante

Le sujet impose une limite de 500 Ko/s par machine du LAN. Cette limite a été mise en œuvre avec tc (traffic control) et une file d'attente hiérarchique HTB (Hierarchical Token Bucket) sur l'interface LAN (eth2) du pare-feu.

```
tc qdisc add dev eth2 root handle 1: htb default 30
tc class add dev eth2 parent 1: classid 1:1 htb rate 500kbps
tc class add dev eth2 parent 1:1 classid 1:30 htb rate 500kbps ceil 500kbps
```

Note d'unité : tc affiche les débits en bits/seconde. La valeur configurée « 500kbps » (500 kilo-octets/s, unité utilisateur de tc) correspond à un affichage de 4 Mbit/s dans tc class show, ce qui est cohérent ($500 \times 8 = 4000$ kbit/s).

Test de validation — téléchargement d'un fichier de 10 Mo depuis le client LAN :

```
root@Client-LAN:/# curl -o /dev/null http://192.168.2.10/testfile.bin
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100 10.0M    100 10.0M    0     0   197k      0  0:00:51  0:00:51 --:--:--  474k
```

La vitesse instantanée en fin de transfert (474 Ko/s) est très proche du plafond configuré (500 Ko/s). La moyenne globale plus basse (197 Ko/s) s'explique par le slow start TCP : le débit réel se stabilise progressivement au fil de la connexion et ne reflète le régime de croisière qu'après les premières secondes.

6. Surveillance IDS avec Snort

Snort (version 2.9.15.1) a été déployé sur le pare-feu en écoute sur l'interface eth0 (WAN), afin de détecter les tentatives de scan de ports provenant de l'extérieur, conformément au sujet.

6.1 Fichier de configuration snort.conf

```
ipvar HOME_NET [192.168.2.0/24,192.168.3.0/24]
ipvar EXTERNAL_NET any

include /etc/snort/rules/local.rules

output alert_fast: /var/log/snort/alert.log
```

6.2 Règles locales (local.rules)

```
alert tcp any any -> $HOME_NET any (msg:"Possible scan de ports detecte"; flags:S;
  threshold: type threshold, track by_src, count 5, seconds 10; sid:1000001; rev:1;)

alert icmp any any -> $HOME_NET any (msg:"Ping ICMP detecte"; sid:1000002; rev:1;)
```

La règle SID 1000001 déclenche une alerte lorsque plus de 5 paquets TCP avec le flag SYN (début de connexion) proviennent de la même source vers le réseau protégé (HOME_NET) en moins de 10 secondes — une signature caractéristique d'un scan de ports.

6.3 Commande de démarrage

```
snort -c /etc/snort/snort.conf -i eth0 -A fast -l /var/log/snort &
```

6.4 Simulation d'un scan de ports et journal d'alerte

Un scan a été simulé depuis Client-WAN par des tentatives de connexion rapprochées vers plusieurs ports du serveur web de la DMZ (20, 21, 22, 23, 25, 80, 443, 8080), reproduisant le comportement typique d'un outil de scan de type nmap :

```
root@Client-WAN:/# for port in 20 21 22 23 25 80 443 8080; do
  timeout 1 bash -c "echo > /dev/tcp/192.168.2.10/$port" 2>&1
done
```

Alerte générée dans le journal Snort (/var/log/snort/alert) :

```
08/14-05:19:50.602998  [**] [1:1000001:1] Possible scan de ports detecte [**]
  [Priority: 0] {TCP} 192.168.1.10:54370 -> 192.168.2.10:25
```

6.5 Analyse de l'alerte

- SID 1:1000001:1 : identifie précisément la règle déclenchée (scan de ports, révision 1).
- Source 192.168.1.10 : correspond à Client-WAN, confirmant que Snort surveille bien le trafic entrant côté WAN.
- Destination 192.168.2.10:25 : le serveur web de la DMZ, sur le port 25 (SMTP), l'un des ports testés lors du balayage.
- Horodatage précis (05:19:50) : permet de corréler l'alerte avec les actions effectuées côté client au même instant.

Cette alerte démontre que Snort détecte correctement un comportement de reconnaissance réseau (scan de ports) émanant du WAN vers la DMZ, conformément à l'exigence du sujet.

7. Résultats des tests — synthèse

Le tableau ci-dessous récapitule l'ensemble des tests de validation réalisés au cours du TP, couvrant le filtrage iptables, le NAT, le filtrage de contenu Squid, la limitation de bande passante et la détection Snort.

Test	Résultat attendu	Résultat obtenu	Statut
WAN → page web DMZ (HTTP)	Accessible	HTTP 200 OK, page Nginx reçue	Conforme
WAN → LAN (ping)	Bloqué	100% de perte (3/3 paquets perdus)	Conforme
LAN → Internet (WAN)	Accessible	0% de perte, ping réussi	Conforme
LAN → DMZ (HTTP)	Accessible (via proxy Squid)	HTTP 200 OK, en-têtes Squid présents	Conforme
LAN → DMZ (ping ICMP)	Bloqué (non listé dans les règles)	100% de perte	Conforme
Serveur Web → Internet / LAN	Bloqué (ne peut sortir)	100% de perte dans les deux sens	Conforme
Accès Facebook/YouTube hors heures de bureau	Autorisé	HTTP 200 OK via Squid	Conforme
Accès Facebook/YouTube en heures de bureau	Bloqué	ERR_ACCESS_DENIED (Squid)	Conforme
Accès site pour adultes (liste noire)	Bloqué en permanence	ERR_ACCESS_DENIED (Squid)	Conforme
Limitation de bande passante LAN	≈ 500 Ko/s par machine	≈ 474 Ko/s mesurés (curl)	Conforme
Détection scan de ports (Snort)	Alerte générée	Alerte SID 1000001 loggée	Conforme

8. Analyse et conclusion

8.1 Analyse

L'architecture mise en place répond à l'ensemble des exigences fonctionnelles du sujet. Le choix d'une politique de filtrage par défaut restrictive (DROP), combinée à des règles ACCEPT explicites ajoutées et validées avant le verrouillage final, s'est révélé être une méthodologie sûre : elle a permis de tester chaque étape sans risquer de perdre l'accès aux machines en cours de configuration.

La principale difficulté technique rencontrée a concerné l'environnement d'exécution : les conteneurs Docker, une fois intégrés à la topologie GNS3 et donc isolés du réseau de l'hôte, ne pouvaient plus télécharger de paquets via apt. La solution retenue — construire une image Docker personnalisée en amont, avec tous les outils nécessaires pré-installés (iproute2, iptables, squid, snort, nginx) — a permis de contourner cette limitation de façon reproductible pour l'ensemble des nœuds du réseau.

Un second point d'attention a été la configuration du mode interception de Squid, qui nécessite un port additionnel non-intercepté (3129) pour son fonctionnement interne (génération des pages d'erreur), en plus du port 3128 utilisé pour le proxy transparent (mode intercept).

Enfin, l'absence de connectivité Internet réelle dans l'environnement de simulation a nécessité une adaptation méthodologique pour tester le filtrage par nom de domaine (Facebook, YouTube, sites pour adultes) : des entrées statiques dans /etc/hosts ont permis de simuler la résolution DNS de ces domaines vers des adresses internes, tout en conservant la validité du test des règles Squid elles-mêmes.

8.2 Conclusion

Ce TP a permis de mettre en pratique l'ensemble de la chaîne de défense périmétrique d'un système d'information : segmentation réseau (WAN/DMZ/LAN), filtrage par pare-feu avec politique de moindre privilège, translation d'adresses (NAT), filtrage de contenu applicatif (proxy Squid avec restrictions horaires et par catégorie), maîtrise de la qualité de service (limitation de bande passante) et détection d'intrusion (Snort).

Chaque brique a été testée individuellement et en interaction avec les autres (par exemple, la redirection transparente iptables + Squid, ou l'articulation entre routage IP et politique de filtrage FORWARD), avec des preuves concrètes (logs, captures de commandes) à l'appui de chaque validation. L'architecture finale correspond fidèlement au schéma DMZ demandé et satisfait l'ensemble des tests requis par l'énoncé.

Annexe — Fichiers de configuration complets

A. Règles iptables complètes (iptables-save)

```
# Generated by iptables-save v1.8.7 on Sun Aug 16 11:06:58 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [11:804]
:OUTPUT ACCEPT [0:0]
-A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -i eth0 -o eth1 -p tcp -m tcp --dport 80 -j ACCEPT
-A FORWARD -i eth0 -o eth1 -p tcp -m tcp --dport 443 -j ACCEPT
-A FORWARD -i eth0 -o eth1 -p tcp -m tcp --dport 22 -j ACCEPT
-A FORWARD -i eth2 -o eth1 -p tcp -m tcp --dport 80 -j ACCEPT
-A FORWARD -i eth2 -o eth1 -p tcp -m tcp --dport 443 -j ACCEPT
-A FORWARD -i eth2 -o eth1 -p tcp -m tcp --dport 22 -j ACCEPT
-A FORWARD -i eth2 -o eth0 -j ACCEPT
-A FORWARD -i eth0 -o eth2 -j DROP
COMMIT
# Completed on Sun Aug 16 11:06:58 2026
# Generated by iptables-save v1.8.7 on Sun Aug 16 11:06:58 2026
*nat
:PREROUTING ACCEPT [0:0]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A PREROUTING -i eth2 -p tcp -m tcp --dport 80 -j REDIRECT --to-ports 3128
-A POSTROUTING -s 192.168.3.0/24 -o eth0 -j MASQUERADE
COMMIT
# Completed on Sun Aug 16 11:06:58 2026
```

B. squid.conf

```
http_port 3129
http_port 3128 intercept
acl localnet src 192.168.3.0/24
acl heures_bureau_matin time MTWHF 08:00-12:00
acl heures_bureau_apresmidi time MTWHF 14:00-18:00
acl reseaux_sociaux dstdomain .facebook.com .youtube.com
acl sites_adultes dstdomain "/etc/squid/blacklist_adultes.txt"
http_access deny sites_adultes
http_access deny reseaux_sociaux heures_bureau_matin
http_access deny reseaux_sociaux heures_bureau_apresmidi
http_access allow localnet
http_access allow localhost
http_access deny all
```

C. snort.conf et local.rules

```
# snort.conf
ipvar HOME_NET [192.168.2.0/24,192.168.3.0/24]
```

```

ipvar EXTERNAL_NET any
include /etc/snort/rules/local.rules
output alert_fast: /var/log/snort/alert.log

# local.rules
alert tcp any any -> $HOME_NET any (msg:"Possible scan de ports detecte"; flags:S;
  threshold: type threshold, track by_src, count 5, seconds 10; sid:1000001; rev:1;)
alert icmp any any -> $HOME_NET any (msg:"Ping ICMP detecte"; sid:1000002; rev:1;)

```

D. Script de configuration IP et pare-feu (script complet)

```

#!/bin/bash
# --- Adressage IP (Firewall-3if-1) ---
ip addr add 192.168.1.1/24 dev eth0
ip addr add 192.168.2.1/24 dev eth1
ip addr add 192.168.3.1/24 dev eth2
ip link set eth0 up ; ip link set eth1 up ; ip link set eth2 up
echo 1 > /proc/sys/net/ipv4/ip_forward

# --- Filtrage FORWARD ---
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 443 -j ACCEPT
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 22 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 443 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 22 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth0 -j ACCEPT
iptables -A FORWARD -i eth0 -o eth2 -j DROP

# --- NAT et proxy transparent ---
iptables -t nat -A POSTROUTING -o eth0 -s 192.168.3.0/24 -j MASQUERADE
iptables -t nat -A PREROUTING -i eth2 -p tcp --dport 80 -j REDIRECT --to-port 3128

# --- Limitation de bande passante (500 Ko/s sur le LAN) ---
tc qdisc add dev eth2 root handle 1: htb default 30
tc class add dev eth2 parent 1: classid 1:1 htb rate 500kbps
tc class add dev eth2 parent 1:1 classid 1:30 htb rate 500kbps ceil 500kbps

# --- Verrouillage final (à exécuter en dernier) ---
iptables -P FORWARD DROP

```